

A Temporal Convolutional Network-based Approach for Network Intrusion Detection

Rukmini Nazre
Department of CSE,
School of Computational Sciences
COEP Technological University
nazrerukmini@gmail.com

Rujuta Budke
Department of CSE,
School of Computational Sciences
COEP Technological University
rujutabudke@gmail.com

Omkar Oak
Department of CSE,
School of Computational Sciences
COEP Technological University
omkarsoak@gmail.com

Suraj Sawant
Department of CSE,
School of Computational Sciences
COEP Technological University
sts.comp@coeptech.ac.in

Amit Joshi
Department of CSE,
School of Computational Sciences
COEP Technological University
adj.comp@coeptech.ac.in

Abstract—Network intrusion detection is critical for securing modern networks, yet the complexity of network traffic poses significant challenges to traditional methods. This study proposes a Temporal Convolutional Network (TCN) model featuring a residual block architecture with dilated convolutions to capture dependencies in network traffic data while ensuring training stability. The TCN's ability to process sequences in parallel enables faster, more accurate sequence modeling than Recurrent Neural Networks. Evaluated on the Edge-IIoTset dataset, which includes 15 classes with normal traffic and 14 cyberattack types, the proposed model achieved an accuracy of 96.72% and a loss of 0.0688, outperforming 1D CNN, CNN-LSTM, CNN-GRU, CNN-BiLSTM, and CNN-GRU-LSTM models. A class-wise classification report, encompassing metrics such as recall, precision, accuracy, and F1-score, demonstrated the TCN model's superior performance across varied attack categories, including Malware, Injection, and DDoS. These results underscore the model's potential in addressing the complexities of network intrusion detection effectively.

Index Terms—Deep Learning, Temporal Convolution Networks, Network Intrusion Detection System, Recurrent Neural Networks, Multiclass Classification, Network Security

I. INTRODUCTION

The rapid growth of IoT devices and edge computing has transformed network topologies, creating complex systems for data collection and processing near the source to improve smart city applications' efficiency. This shift to edge computing distributes computational resources closer to data sources but also brings significant security risks. The expanded attack surface of IoT networks makes them prime targets for cyberattacks, highlighting the need for strong intrusion detection systems (IDS) to protect sensitive information [1]. These networks face an array of sophisticated cyber threats, including malware intrusions, Distributed Denial of Service (DDoS) attacks, and Man-In-The-Middle (MITM) exploits [2].

Previous research has extensively explored various approaches to address these security challenges. Classical Machine Learning (ML) algorithms, such as k-Nearest

Neighbors (KNN), Support Vector Machines (SVM), and Random Forests (RF), have been implemented for intrusion detection [3]–[5]. However, these traditional methods face several limitations in terms of their adaptability to evolving attack patterns, insufficient capability to handle heterogeneous traffic patterns, reduced effectiveness in processing temporal dependencies in network traffic, and challenges in real-time detection and scalability. Motivated by these challenges, this research aims to develop a more robust and efficient intrusion detection system specifically designed for edge computing environments. Our primary objectives include designing a novel IDS framework utilizing Temporal Convolution Networks (TCNs), evaluating their effectiveness in capturing sequential patterns within network traffic, comparing their performance against traditional CNN architectures, and validating the system's effectiveness using the comprehensive Edge-IIoTset dataset [6].

This work makes several significant contributions to the field of network security in edge computing environments. First, we develop a novel TCN-based architecture specifically optimized for network intrusion detection. Second, we provide a comprehensive comparative analysis of the proposed TCN model against conventional CNN architectures. Third, we conduct empirical validation using the Edge-IIoTset dataset, demonstrating superior detection accuracy and reduced false positive rates. Thus, we present a scalable framework that can be deployed across diverse edge computing scenarios.

The remainder of this paper is organized as follows. Section II presents a comprehensive review of related work in network intrusion detection. Section III details the proposed TCN-based methodology. Section IV presents the experimental setup, evaluation measures, results and comparative analysis. Finally, Section V concludes the paper and discusses future research directions.

II. RELATED WORKS

Traditional ML models are not used in Network Intrusion Detection as they fail to capture intricate patterns from complex network traffic. So, more advanced Deep Learning (DL) approaches have been developed, demonstrating significantly improved performance. CNNs are frequently used for network intrusion detection [7]. By using convolutional layers, 1D CNN models are able to capture patterns in network traffic, such as the frequency of certain types of packets or anomalies. However, CNNs lack the ability to capture temporal dependencies, that are critical for analyzing network traffic sequences. To address this, hybrid models that combine CNN with recurrent networks including Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), have been explored. GRUs and LSTMs are designed to handle time-series data by maintaining a memory of earlier inputs, making them effective for sequential data analysis [8], [9]. The use of Bidirectional LSTM (BiLSTM) further enhances temporal modeling by considering the context from both past and future inputs [10]. Hybrid models like CNN-BiLSTM and CNN-GRU have demonstrated strong performance in various anomaly detection tasks but come with higher computational complexity and longer training times, especially on large datasets [11]. Most of these models focus on specific attack types or simplified network environments, leaving room for more comprehensive solutions. Recent advancements in DL have introduced TCNs [12], which offer an alternative to recurrent models. TCNs use one dimensional convolutional layers with dilated convolutions that help capture dependencies over a long range in data, enabling them to process sequential information in a parallelized manner. This architecture mitigates the common issues in recurrent networks, such as vanishing gradients and slow training times [13]. Many studies have explored the usefulness of TCNs for time series prediction tasks, but their application in network intrusion detection still remains relatively new. The ability of TCNs to handle sequential data without relying on recurrence makes them particularly suitable for processing network traffic data, which can exhibit temporal patterns of normal and abnormal behavior. Their use provides an opportunity to enhance detection accuracy while maintaining faster processing times compared to traditional recurrent networks. Existing work on IDS in IoT environments has been centered primarily around specific attacks like DDoS [14] using datasets such as IoT-23 [15] and CIC-IDS2018 [16]. These datasets, while useful, do not capture the comprehensive nature of IoT threats. The Edge-IIoTset dataset [6] is a comprehensive benchmark for evaluating the performance of various models on 14 different types of attacks. The dataset's rich collection of various types of attacks, such as malware, injection, MITM and scanning, has made it a reliable resource for testing the performance of various DL models [17], [18]. Existing studies have shown that while CNN-LSTM and CNN-GRU are effective at detecting DDoS attacks, they struggle with more complex

attack vectors such as MITM or malware due to their reliance on temporal sequence modeling [19]. TCNs offer a promising solution for this task [20], [21]. They have been adopted for use on other datasets such as the APA-DDoS Dataset [22] and the Bot-IoT Dataset [23]. They have also been used for Binary Classification on the Edge-IIoTset dataset [24]. However, there is a lack of studies focusing on TCN-based frameworks for Multiclass Classification on the EdgeIoTset dataset. This study proposes a TCN based model for 15-class classification on the EdgeIoTset dataset, and compares its performance with 1D CNN, CNN-LSTM, CNN-GRU, CNN-BiLSTM and CNN-GRU-LSTM.

III. PROPOSED METHOD

This section focuses on model development and dataset preparation. It covers the implementation of CNN, hybrid CNN models, and the proposed TCN model followed by a detailed description of the dataset, data preprocessing techniques, and the classification of output attack types.

A. Prediction models

1) *One-directional CNN*: 1D CNNs [25] process sequential data by applying convolutional filters along the temporal dimension, efficiently extracting local features that are useful for sequential data applications like language modeling, speech recognition, or time series analysis. However, their limited receptive field restricts their ability to capture long-range dependencies in the sequence, making it difficult to identify relationships between distant timesteps.

2) *Hybrid CNN*: Hybrid models like CNN-GRU, CNN-LSTM, and CNN-BiLSTM enhance 1D CNNs by integrating recurrent layers to better capture temporal dependencies. CNN-GRU offers a balance of performance and efficiency, while CNN-LSTM is effective for long-term dependencies but with higher computational costs. CNN-BiLSTM processes sequences bidirectionally, improving context understanding but increasing training time. Among these, CNN-LSTM is often preferred for its robust long-term dependency management, despite its complexity. While hybrid models improve upon traditional 1D CNNs, their sequential nature limits parallelization and increases overfitting risks.

3) *Proposed Model*: The proposed model, shown in Figure 1, employs TCN to overcome the limitations of hybrid models like CNN-LSTM. TCNs utilize causal convolutions to ensure predictions depend only on past data, preserving temporal order and enabling efficient parallel processing. Dilated convolutions expand the receptive field without increasing network depth, allowing TCNs to effectively capture both short and long range dependencies. TCN architecture has residual connections, which improve gradient flow and mitigate vanishing gradient issues, stabilizing training and enhancing representation learning.

The proposed model consists of three stacked residual blocks with varying dilation rates to capture dependencies across different time scales. Following these blocks, the output

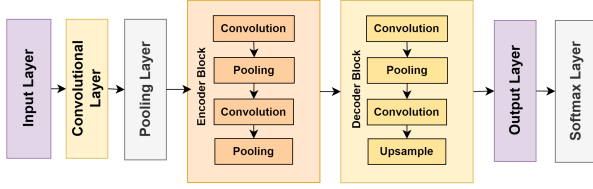


Fig. 1. Architecture of the Proposed Model

is flattened, processed through a fully connected layer containing 128 neurons with dropout for regularization, and concludes with a softmax layer for classification. By integrating parallel computation, dilated convolutions, and residual connections, the proposed TCN model effectively manages long-term dependencies while minimizing computational costs.

B. Dataset

This study utilizes the publicly available Edge-IIoTset dataset [6], specifically the DNN-EdgeIIoT dataset, which consists of a CSV file containing 61 features along with two label columns— ‘Attack_Label’ and ‘Attack_Type’. The dataset simulates 14 types of cyberattacks, classified into the following major subcategories: Information gathering, MITM, DoS/DDoS, Malware, and Injection attacks. The dataset includes features enabling identification of network intrusion patterns such as TCP control flags, HTTP request types, ICMP sequence numbers, DNS query names, and MQTT topics.

TABLE I
LIST OF ALL LABEL CLASSES

Attack Type	Support Count
Normal	349906
DDoS_UDP	30392
DDoS_ICMP	16985
SQL_injection	12706
DDoS_TCP	12515
Vulnerability_scanner	12507
Password	12483
DDoS_HTTP	12136
Uploading	9239
Backdoor	6007
Port_Scanning	4994
XSS	3767
Ransomware	2422
Fingerprinting	213
MITM	90

This study uses the dataset for multiclass classification, 15 classes specifically, which are shown in Table I. After preprocessing, the dataset contains 4,86,362 entries. A train, test and validation split of 70%, 20% and 10% respectively is used. The training dataset has 3,40,452 values, test 97,273 and validation 48,637.

C. Data Preprocessing

1) *Encoding and Size Reduction*: Label encoding is applied to categorical features related to HTTP requests, DNS queries, and MQTT protocol fields. Figure II contains all of the features used for classification. The features are one-hot encoded to prepare them for model training. Duplicate rows are identified and removed to maintain data integrity. Additionally, a hash-based method is used to find columns with identical content,

TABLE II
LIST OF INPUT FEATURES

Feature	Feature	Feature
1. tcp.ack	32. http1_2	63. dns_6
2. tcp.ack_raw	33. http.response	64. mqtt3_1
3. icmp.transmit_timestamp	34. dns_0	65. mqtt2_1
4. tcp.seq	35. mqtt3_0	66. mbtcp.transmit_request
5. udp.stream	36. mqtt1_0	67. mbtcp.transmit_request_in
6. dns.qry.name	37. mqtt2_0	68. dns_4
7. icmp.checksum	38. tcp.connection.syn	69. dns_6
8. icmp.seq_le	39. http3_5	70. http1_7
9. tcp.dport	40. http1_1	71. http3_7
10. tcp.checksum	41. tcp.flags.ack	72. http3_3
11. mqtt.hdrflags	42. http1_4	73. http3_10
12. tcp.len	43. http1_encoded	74. dns_9
13. http.content_length	44. http2_encoded	75. dns_5
14. udp.time_delta	45. http2_2	76. mqtt1_1
15. mqtt.len	46. mqtt.proto_len	77. http3_1
16. mqtt1.len	47. mqtt.conf.flags	78. http1_8
17. tcp.flags	48. http2_3	79. dns.retransmit_request
18. http1_2	49. mqtt3_encoded	80. mqtt1_11
19. mqtt1_1	50. http3_2	81. dns_8
20. http1_0	51. tcp.connection.fin	82. mqtt3_2
21. mqtt1_encoded	52. http4_4	83. http5_1
22. mqtt3_2	53. http4_5	84. http5_2
23. dns_1	54. mqtt.conf.cleaness	85. mqtt1_3
24. dns_encoded	55. mqtt1_2	86. http2_2
25. dns_3	56. mqtt3_2	87. mqtt1_4
26. mqtt3_encoded	57. mqtt2_2	88. http5_1
27. mqtt2_encoded	58. dns.retransmit	89. http3_2
28. mqtt2_encoded	59. mbtcp.trans_id	90. dns_7
29. mqtt.topic_len	60. mbtcp.unit_id	91. http3_5
30. mqtt.msgtype	61. mbtcp.func_id	92. mqtt1_5
31. http1_0	62. http3_12	

and the duplicate columns are dropped. The Chi-Squared test is employed to identify and rank features based on their significance in relation to the target variable. The Chi-Squared test is a statistical test that evaluates the relationship between two categorical variables. It is chosen for feature selection because it can help determine which features are most strongly associated with the target variable - ‘Attack_type’. By ranking the features based on their Chi-Squared statistic, the most informative features can be identified and selected for model training. Certain columns irrelevant to the analysis, such as timestamps, IP addresses, as well as specific protocol data fields are removed. The distribution of different attack types is confirmed through an inspection of the attack type labels, and the redundant icmp.unused column is removed for its lack of meaningful contribution. To optimize the dataset for model training, a size reduction is implemented using stratified sampling. The dataset is reduced by a factor of 0.25, ensuring that the class distribution of the ‘Attack_type’ column remains consistent.

2) *Feature Scaling*: In this stage, feature scaling is performed using standardization to ensure that the input features have a standard deviation equal to one and a mean equal to zero. The StandardScaler is fitted to the training set, and subsequently applied to transform the test, validation, and training sets. This helps to mitigate issues related to differing scales of the features, thereby improving the performance and convergence speed of the models. By standardizing the data, this study ensures that each feature contributes equally to the distance calculations in algorithms that rely on distance metrics.

D. Output Classes

The dataset includes both normal and malicious network traffic, categorized into various attack types. Normal traffic corresponds to communication or data exchanges which are non-malicious and legitimate. DDoS attacks flood systems

with excessive traffic, preventing normal operations. These include DDoS attacks targeting different network protocols such as ICMP, UDP, TCP and HTTP. SQL Injection allows attackers to manipulate databases through malicious queries, while Uploading involves uploading harmful files to compromise systems. Cross-Site Scripting (XSS) injects malicious scripts into web pages, leading to data theft or session hijacking. Vulnerability Scanners and Port Scanning detect system weaknesses, while Fingerprinting gathers system data for exploitation. Password Attacks focus on unauthorized access by cracking credentials, and Malware includes Backdoor attacks, which bypass authentication, and Ransomware, which encrypts files for ransom. Lastly, MITM attacks attempt to alter as well as intercept communications taking place amongst two entities. These attack types highlight key methods by which systems and networks can be compromised, essential for understanding cybersecurity threats.

IV. RESULTS AND DISCUSSION

This section discusses the experimental setup, evaluation measures and results, as well as a comparison with existing research.

A. Experimental Setup

The models are trained and tested in identical environments using Google Colab, equipped with an NVIDIA T4 GPU accelerator. The software requirements consist of Python 3.10 or a higher version, along with TensorFlow 2.17. The models are trained for 5 epochs using a learning rate of 0.001 and a batch size of 32. The Adam optimizer is used, and the loss function is Sparse Categorical Crossentropy.

B. Evaluation Measures

1) *Overall Accuracy*: Overall accuracy represents the percentage of correct predictions made by the model on the dataset, as shown in Equation 1.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Where TP , TN , FP , and FN represent true positives, true negatives, false positives, and false negatives, respectively.

2) *Overall Loss - Sparse Categorical Cross Entropy*: This loss function measures the difference between actual class labels and predicted probabilities in a multiclass classification model that uses class indices instead of one-hot encoding, as defined in Equation 2.

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N \log(p_{y_i}) \quad (2)$$

where N is the number of samples, y_i is the true class label for sample i , and p_{y_i} is the predicted probability of the true class.

3) *Classification Report*: The classification report provides metrics for each class, including:

i) *Precision*: The number of positive predictions that are correct divided by the number of positive predictions overall, indicating accuracy, as shown in Equation 3.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

ii) *Recall*: The number of positives predicted correctly divided by the number of actual positives, showing how well the model identifies positive cases, as defined in Equation 4.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

iii) *F1 Score*: A metric providing a balanced performance measure between recall and precision, as described in Equation 5.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

iv) *Support*: The number of actual occurrences of each class, showing how classes are distributed.

C. Results and Comparison

As shown in Table III, the proposed TCN model outperforms all other architectures in both accuracy and loss. The TCN achieves the highest test accuracy of 96.72% and the lowest test loss of 0.0668, demonstrating its superior ability to capture complex patterns and dependencies in the data. In contrast, the 1D CNN has an accuracy of 96.18%, the lowest among all evaluated models and the highest test loss of 0.0760, suggesting that it struggles to fully capture the intricate relationships within the dataset.

TABLE III
PERFORMANCE OF DIFFERENT MODELS

Model	Test Accuracy	Test Loss
1D CNN	0.9618	0.0760
CNN GRU	0.9638	0.0732
CNN LSTM	0.9635	0.0739
CNN BiLSTM	0.9640	0.0756
CNN LSTM GRU	0.9640	0.0733
TCN	0.9672	0.0668

Models incorporating recurrent layers such as CNN-GRU, CNN-LSTM, and CNN-LSTM-GRU improve accuracy and reduce loss compared to the standalone CNN. CNN-LSTM-GRU matches the performance of CNN-BiLSTM at 96.40% accuracy. However, none of these models surpass the performance of the TCN. While existing research has utilized TCNs in various contexts, none have specifically addressed the challenge of 15-way multiclass classification on the Edge-IIoTset dataset [24]. This study demonstrates significant advancements over existing research in network intrusion detection on this dataset [6].

The classification reports for the models in Tables IV, V, VI, VII, VIII and IX and the confusion matrix of the proposed model in Figure 2 show that all models perform well on the highly represented 'Normal' class. However, their performance on classes with lower representation such as 'SQL_injection'

and ‘XSS’, is poor. The TCN architecture demonstrates robust performance across both high-frequency and low-frequency class distributions, particularly excelling in classes like ‘SQL injection’ and ‘Uploading’ with a weighted F1-score of 0.97.

The proposed model outperforms existing models due to the following key improvements. By using stacked residual blocks with varying dilation rates, the model captures dependencies across different time scales more effectively. The integration of dilated convolutions allows the model to process both immediate and distant patterns. Additionally, the residual connections facilitate the training of deeper networks, preventing gradient problems. With the incorporation of parallel computation, the model significantly enhances training efficiency, while the fully connected layer and softmax classification ensure robust performance on large datasets.

TABLE IV
1D CNN CLASSIFICATION REPORT

	Precision	Recall	F1-Score	Support
Normal	1	1	1	69832
MITM	1	0.94	0.97	18
Uploading	0.86	0.64	0.73	1872
Ransomware	0.93	0.97	0.95	479
SQL_injection	0.54	1	0.70	2500
DDoS_HTTP	0.94	0.88	0.91	2507
DDoS_TCP	0.98	0.92	0.95	2500
Password	0.93	0.33	0.49	2507
Port_Scanning	0.83	1	0.91	1010
Vulnerability_scanner	0.98	0.91	0.95	2495
Backdoor	1	0.95	0.97	1221
XSS	0.62	0.89	0.73	755
Fingerprinting	1	0.36	0.53	39
DDoS_UDP	1	1	1	6005
DDoS_ICMP	0.99	1	1	3533
Accuracy			0.97	97273
Macro avg	0.91	0.85	0.85	97273
Weighted avg	0.98	0.97	0.97	97273

TABLE V
CNN-GRU CLASSIFICATION REPORT

	Precision	Recall	F1-Score	Support
Normal	1.00	1.00	1.00	69832
MITM	1.00	1.00	1.00	18
Uploading	0.88	0.52	0.66	1872
Ransomware	0.92	0.97	0.95	479
SQL_injection	0.76	0.59	0.66	2500
DDoS_HTTP	0.89	0.90	0.89	2507
DDoS_TCP	0.98	0.92	0.95	2500
Password	0.50	0.77	0.61	2507
Port_Scanning	0.84	1.00	0.91	1010
Vulnerability_scanner	0.96	0.92	0.94	2495
Backdoor	1.00	0.95	0.97	1221
XSS	0.60	0.67	0.64	755
Fingerprinting	1.00	0.41	0.58	39
DDoS_UDP	1.00	1.00	1.00	6005
DDoS_ICMP	1.00	1.00	1.00	3533
Accuracy			0.96	97273
Macro avg	0.89	0.84	0.85	97273
Weighted avg	0.97	0.96	0.96	97273

V. CONCLUSION

This study proposes a novel TCN-based model for network intrusion detection, demonstrating strong performance on the Edge-IIoTset dataset. The TCN demonstrated superior performance, achieving the highest test accuracy of 96.72% and the lowest test loss of 0.0668 compared to other existing models. The classification reports further highlight the

TABLE VI
CNN-LSTM CLASSIFICATION REPORT

	Precision	Recall	F1-Score	Support
Normal	1.00	1.00	1.00	69832
MITM	1.00	1.00	1.00	18
Uploading	0.93	0.58	0.71	1872
Ransomware	0.79	0.90	0.84	479
SQL_injection	0.70	0.66	0.68	2500
DDoS_HTTP	0.79	0.96	0.87	2507
DDoS_TCP	0.97	0.93	0.95	2500
Password	0.54	0.72	0.61	2507
Port_Scanning	0.85	1.00	0.92	1010
Vulnerability_scanner	0.96	0.92	0.94	2495
Backdoor	0.98	0.88	0.93	1221
XSS	0.78	0.33	0.46	755
Fingerprinting	1.00	0.18	0.30	39
DDoS_UDP	1.00	1.00	1.00	6005
DDoS_ICMP	0.99	1.00	1.00	3533
Accuracy			0.96	97273
Macro avg	0.89	0.80	0.81	97273
Weighted avg	0.97	0.96	0.96	97273

TABLE VII
CNN-BiLSTM CLASSIFICATION REPORT

	Precision	Recall	F1-Score	Support
Normal	1.00	1.00	1.00	69832
MITM	1.00	0.89	0.94	18
Uploading	0.81	0.55	0.66	1872
Ransomware	0.94	0.90	0.92	479
SQL_injection	0.56	0.87	0.68	2500
DDoS_HTTP	0.94	0.87	0.90	2507
DDoS_TCP	0.97	0.94	0.95	2500
Password	0.65	0.44	0.53	2507
Port_Scanning	0.87	0.96	0.91	1010
Vulnerability_scanner	0.96	0.92	0.94	2495
Backdoor	0.97	0.96	0.96	1221
XSS	0.61	0.85	0.71	755
Fingerprinting	1.00	0.36	0.53	39
DDoS_UDP	1.00	1.00	1.00	6005
DDoS_ICMP	0.99	1.00	1.00	3533
Accuracy			0.96	97273
Macro avg	0.88	0.83	0.84	97273
Weighted avg	0.97	0.96	0.96	97273

TABLE VIII
CNN-LSTM-GRU CLASSIFICATION REPORT

	Precision	Recall	F1-Score	Support
Normal	1.00	1.00	1.00	69832
MITM	1.00	0.89	0.94	18
Uploading	0.80	0.68	0.74	1872
Ransomware	0.82	0.78	0.80	479
SQL_injection	0.54	0.98	0.70	2500
DDoS_HTTP	0.91	0.90	0.90	2507
DDoS_TCP	0.98	0.92	0.95	2500
Password	0.94	0.31	0.46	2507
Port_Scanning	0.84	1.00	0.91	1010
Vulnerability_scanner	0.98	0.92	0.95	2495
Backdoor	0.93	0.90	0.91	1221
XSS	0.60	0.73	0.66	755
Fingerprinting	1.00	0.36	0.53	39
DDoS_UDP	1.00	1.00	1.00	6005
DDoS_ICMP	0.99	1.00	1.00	3533
Accuracy			0.96	97273
Macro avg	0.89	0.82	0.83	97273
Weighted avg	0.97	0.96	0.96	97273

TCN’s effectiveness in accurately identifying a wide range of attack types, particularly excelling in detecting less frequent and more complex threats like SQL injection and uploading attacks. The enhanced performance of the TCN can be attributed to its ability to capture long-range dependencies and efficiently model temporal patterns within the network traffic data. Overall, the TCN-based model proves to be a robust and reliable solution for improving network security

TABLE IX
PROPOSED TCN MODEL CLASSIFICATION REPORT

	Precision	Recall	F1-Score	Support
Normal	1.00	1.00	1.00	69832
MITM	1.00	1.00	1.00	18
Uploading	0.86	0.67	0.75	1872
Ransomware	0.92	0.97	0.95	479
SQL_injection	0.55	1.00	0.71	2500
DDoS_HTTP	0.93	0.89	0.91	2507
DDoS_TCP	0.98	0.94	0.96	2500
Password	0.92	0.33	0.48	2507
Port_Scanning	0.87	0.98	0.92	1010
Vulnerability_scanner	0.98	0.92	0.95	2495
Backdoor	1.00	0.95	0.97	1221
XSS	0.61	0.83	0.71	755
Fingerprinting	1.00	0.36	0.53	39
DDoS_UDP	1.00	1.00	1.00	6005
DDoS_ICMP	0.99	1.00	1.00	3533
Accuracy			0.97	97273
Macro avg	0.91	0.86	0.86	97273
Weighted avg	0.98	0.97	0.97	97273

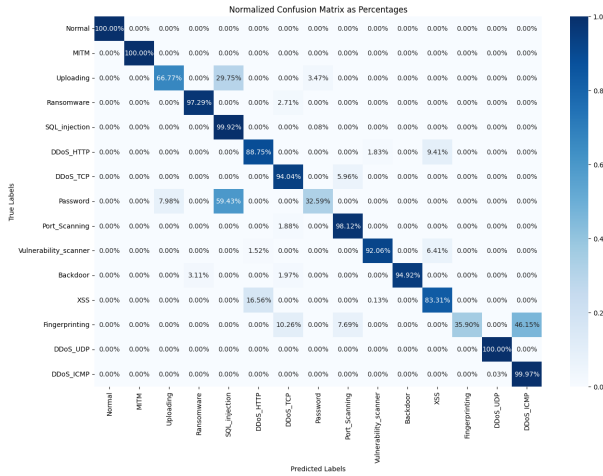


Fig. 2. Confusion Matrix for the Proposed Model

through accurate and efficient intrusion detection. Despite its strong performance, the proposed TCN-based model has certain limitations. It's evaluation is limited to the Edge-IIoTset dataset, which may restrict its generalizability to other datasets or real-time environments. The model's computational complexity could also challenge its deployment in resource-constrained settings. In future work, classification accuracy can be improved, especially for complex attacks. The scalability and performance can also be evaluated in dynamic conditions. Finally, the trust and usability of automated IDS in critical infrastructure can be increased, helping administrators better understand detection decisions.

REFERENCES

- [1] Corallo, A., Lazoi, M., Lezzi, M., & Luperto, A. (2022). Cybersecurity awareness in the context of the Industrial Internet of Things: A systematic literature review. *Computers in Industry*, 137, 103614.
- [2] Abdulganiyu, O. H., Tchakouch, T. A., & Saheed, Y. K. (2024). Towards an efficient model for network intrusion detection system (IDS): systematic literature review. *Wireless Networks*, 30(1), 453-482.
- [3] Gu, J., & Lu, S. (2021). An effective intrusion detection approach using SVM with naïve Bayes feature embedding. *Computers & Security*, 103, 102158.
- [4] Liu, G., Zhao, H., Fan, F., Liu, G., Xu, Q., & Nazir, S. (2022). An enhanced intrusion detection model based on improved kNN in WSNs. *Sensors*, 22(4), 1407.
- [5] Awotunde, J. B., Ayo, F. E., Panigrahi, R., Garg, A., Bhoi, A. K., & Barsocchi, P. (2023). A multi-level random forest model-based intrusion detection using fuzzy inference system for internet of things networks. *International Journal of Computational Intelligence Systems*, 16(1), 31.
- [6] Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., & Janicke, H. (2022). Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. *IEEE Access*, 10, 40281-40306.
- [7] Kilichev, D., & Kim, W. (2023). Hyperparameter optimization for 1D-CNN-based network intrusion detection using GA and PSO. *Mathematics*, 11(17), 3724.
- [8] Wang, Z., Huang, H., Du, R., Li, X., & Yuan, G. (2023). IoT intrusion detection model based on CNN-GRU. *Frontiers in Computing and Intelligent Systems*, 4(2), 90-95.
- [9] Altunay, H. C., & Albayrak, Z. (2023). A hybrid CNN+ LSTM-based intrusion detection system for industrial IoT networks. *Engineering Science and Technology, an International Journal*, 38, 101322.
- [10] Said, R. B., Sabir, Z., & Askerzade, I. (2023). CNN-BiLSTM: A hybrid deep learning approach for network intrusion detection system in software defined networking with hybrid feature selection. *IEEE Access*.
- [11] Gao, J. (2022). Network intrusion detection method combining CNN and BiLSTM in cloud computing environment. *Computational Intelligence and Neuroscience*, 2022, 7272479. <https://doi.org/10.1155/2022/7272479>
- [12] Lea, C., Flynn, M. D., Vidal, R., Reiter, A., & Hager, G. D. (2017). Temporal convolutional networks for action segmentation and detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 156-165).
- [13] Shiri, F. M., Perumal, T., Mustapha, N., & Mohamed, R. A. (2023). A Comprehensive Overview and Comparative Analysis on Deep Learning Models. CNN, RNN, LSTM, GRU.
- [14] Kumari, P., & Jain, A. K. (2023). A comprehensive study of DDoS attacks over IoT network and their countermeasures. *Computers & Security*, 127, 103096.
- [15] Garcia, S., Parmisano, A., & Erquiaga, M. J. (2020). IoT-23: A labeled dataset with malicious and benign IoT network traffic (Version 1.0.0) [Data set]. *Zenodo*. <http://doi.org/10.5281/zenodo.4743746>
- [16] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSP*, 1, 108-116.
- [17] Al Nuaimi, T., Al Zaabi, S., Alyilieli, M., AlMaskari, M., Alblooshi, S., Alhabsi, F., ... & Al Badawi, A. (2023). A comparative evaluation of intrusion detection systems on the edge-IIoT-2022 dataset. *Intelligent Systems with Applications*, 20, 200298.
- [18] Hamza, N., Lakmal, H. K. I. S., Maduranga, M. W. P., & Kathriarachchi, R. P. S. (2023, August). Malware detection of IoT networks using machine learning: An experimental study with Edge-IIoT dataset. In *30th Annual Technical Conference-IET Sri Lanka Network, Colombo, Sri Lanka*.
- [19] Jose, J., & Jose, D. V. (2023). Deep learning algorithms for intrusion detection systems in internet of things using CIC-IDS 2017 dataset. *International Journal of Electrical and Computer Engineering (IJECE)*, 13(1), 1134-1141.
- [20] Shi, D., Xu, M., Wu, T., & Kou, L. (2021). Intrusion detecting system based on temporal convolutional network for in-vehicle CAN networks. *Mobile Information Systems*, 2021(1), 1440259.
- [21] Cheng, P., Xu, K., Li, S., & Han, M. (2022). TCAN-IDS: Intrusion detection system for internet of vehicle using temporal convolutional attention network. *Symmetry*, 14(2), 310.
- [22] Rani, Y. A., & Reddy, E. S. (2024). Deep intrusion net: An efficient framework for network intrusion detection using hybrid deep TCN and GRU with integral features. *Wireless Networks*, 1-24.
- [23] DLiu, Z., Liu, S., & Zhang, J. (2024). An Industrial Intrusion Detection-Method Based on Hybrid Convolutional Neural Networks with Improved TCN. *Computers, Materials & Continua*, 78(1).
- [24] Alotaibi, A., & Barnawi, A. (2024). LightFIDS: Lightweight and Hierarchical Federated IDS for Massive IoT in 6G Network. *Arabian Journal for Science and Engineering*, 49(3), 4383-4399.
- [25] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4), 541-551.